PDF2MD Hybrid VLM Refactor

PDF2MD engineering

2026-08-30

Executive summary

PDF2MD now uses a selective visual-enhancement route rather than replacing structural extraction. The default remains native `pdf_oxide` extraction for text-rich pages. OCR remains the route for scanned or sparse pages. An opt-in cloud vision request is made only for visual candidates: detected visual regions, figures, charts, diagrams, images, tables/formulas, or conservative repeated diagram-label signals.

The raw per-page JSON is authoritative for extraction. Visual interpretation is stored in a companion artifact and is appended to Markdown only after the native content. This prevents a model response from destroying source coordinates, OCR evidence, status fields, or error records.

Architecture and data flow

```
PDF
  -> page probe
  -> pdf_oxide native extraction OR faster-paddle/OCR
  -> page_NNN.json (immutable source record)
  -> candidate classifier
      -> ordinary page: no VLM request
      -> visual candidate: render one PNG at 150 DPI
  -> strict OpenAI-compatible vision request
  -> page_NNN.visual.json
  -> page_NNN.md with provenance section
  -> document.md
  -> Pandoc + XeLaTeX
```

The route is intentionally linear. No provider trait, queue, database, model registry, local VLM installation, or persistent worker was added. The Python helper is a runtime bridge called by Rust; it uses only the standard library and existing Poppler tooling.

Implemented phases

Phase 0: Ponytail review and audit

The audit traced `main.rs`, `page.rs`, `pdfoxide_backend.rs`, `reconstruct.rs`, `report.rs`, `manifest.rs`, and the atomic writer. Existing helpers were reused. The following were rejected as unnecessary for the first vertical slice:

- generic provider abstraction;
- background queue or scheduler;
- database state;
- custom HTTP dependency;
- local 7–8B VLM deployment on a 7.6 GiB CPU-only machine;
- Mermaid generation before graph certainty is measurable;
- sending the complete PDF to a model.

The real dependency boundary is an OpenAI-compatible `/chat/completions` endpoint. The credential is read from the process environment and is never placed in prompts, artifacts, or logs.

Phase 1: Candidate routing

`is_visual_candidate` returns true when one of the following is present:

1. `quality.visual_region_detected`;
2. `visual_object` or `table_detected` risk metadata;
3. a figure/chart/diagram/image/table/formula layout region with sufficient score;
4. repeated native labels that strongly resemble a visual diagram, such as transaction/block and key/signature terminology.

The classifier is conservative and documented with a ceiling: it is a keyword fallback until stable native PDF line/shape geometry is available. It is not used to classify every text page as visual.

Tests prove that ordinary text is skipped while visual flags and figure-like regions are selected.

Phase 2: Rendering

Only one candidate page is rendered. The helper uses `pdftoppm` at 150 DPI, stores no permanent PNG, computes an SHA-256 source-image hash, and deletes the temporary directory automatically. The full PDF is never sent to the endpoint.

The current implementation deliberately uses full-page rendering. Cropping is deferred until the existing layout bounding boxes can be proven accurate for diagram regions; a wrong crop is worse than a larger but complete page image.

Phase 3: Strict visual contract

The model is instructed to return JSON with this shape:

```
{
  "description": "string",
  "labels": [
    {"text": "string", "certainty": "visible|uncertain"}
  ],
  "relationships": [
    {
      "from": "string",
      "to": "string",
      "relation": "string",
      "certainty": "visible|inferred|uncertain"
    }
```

```
  ],
  "uncertainties": ["string"]
}
```

The helper rejects non-object responses, empty descriptions, malformed labels, malformed relationships, and unknown certainty values. It accepts a fenced JSON response only as a transport convenience, then validates the parsed object.

The artifact adds:

- `schema_version: visual-v1`;
- page number;
- `status`;
- method and model;
- source image SHA-256;
- normalized visual fields;
- generated Markdown.

The Markdown explicitly says it is a visual interpretation and does not replace raw page JSON. Mermaid is not generated because a readable but incorrect graph is a higher risk than prose with uncertainty markers.

Phase 4: Client, retries, and errors

The helper sends a Base64 `data:image/png` URL in a multimodal user content array. Retry is bounded to three attempts for network failures, timeout-like URL errors, HTTP 408, HTTP 429, and HTTP 5xx. Backoff is 1, 2, then 4 seconds. HTTP 402 and other 4xx errors stop immediately.

A technical failure writes a companion error JSON where possible and leaves the raw page JSON untouched. Extraction errors and visual uncertainty are separate concepts:

- extraction failure: page JSON `status=error`;
- provider/render/schema failure: visual artifact `status=error`;
- unclear diagram area: `uncertainties` in a successful visual artifact.

One existing security defect was corrected: the legacy Rust request path had a literal redacted authorization header format. It now constructs the header with the process-local key while never printing it.

Phase 5: Cache and efficiency

Existing Markdown caching is checked before any visual request. The cache key includes model, prompt, and page JSON input, so valid reconstructed output avoids a duplicate request. Visual requests are one per candidate page, not one per detected object. Concurrency remains user-controlled and should default to one for cloud-rate-limit and memory safety.

A future cache refinement can include the rendered image hash directly in a visual-specific cache key if PDFs are replaced in place while page JSON remains unchanged. That is not required for the current vertical slice because the visual artifact already records the source-image hash and the page-level reconstruction cache is invalidated with page JSON changes.

Phase 6: Markdown and manifest

Native Markdown remains first. If a visual artifact is available, its generated interpretation is appended after the native page content. Raw page JSON is never overwritten.

The manifest now exposes:

```
vlm_candidates
visual_success
visual_partial
visual_error
visual_skipped
```

Existing counters remain intact:

```
json_success
json_partial
json_error
content_integrity
```

This keeps quality metadata separate from artifact existence and makes partial/error coverage observable.

Phase 7: XeLaTeX report

The report path is:

```
Markdown -> Pandoc -> XeLaTeX -> A4 PDF
```

The reproducible command is:

```
pandoc docs/refactor-plan-vlm.md \
  -o docs/pdf2md-vlm-refactor-report.pdf \
  --pdf-engine=xelatex \
  -V papersize=a4 \
  -V geometry:margin=2cm \
  -V fontsize=11pt \
  -V linestretch=1.15
```

The Pandoc manual documents PDF generation through a LaTeX engine and the use of variables such as `geometry` and `linestretch`. The resulting PDF must be checked with `pdfinfo`, `pdftotext -layout`, and a visual page render. A zero compiler exit code alone is not sufficient for diagram-heavy output.

Internet validation ledger

The implementation decisions were checked against current public documentation:

1. OpenAI vision guide: `https://developers.openai.com/api/docs/guides/images-vision` documents image analysis through Chat Completions and accepts fully qualified image URLs or Base64-encoded data URLs. This validates the request shape used by the helper. The endpoint remains provider-compatible rather than provider-specific.

2. Pandoc manual: `https://pandoc.org/MANUAL.html` documents PDF creation, `--pdf-engine`, LaTeX variables, `geometry`, and `linestretch`. This validates the report command.
3. pypdfium2 documentation: `https://pypi.org/project/pypdfium2/` documents Python bindings to PDFium, pre-built wheels, rendering/inspection support, and optional Pillow/NumPy helpers. PDFium remains a documented alternative renderer, not an extra runtime dependency in this patch.
4. PaddleOCR repository: `https://github.com/PaddlePaddle/PaddleOCR` remains the upstream reference for OCR/document parsing. The local hardware assessment still favors `faster-paddle` for ordinary OCR and cloud VLM for difficult semantic diagrams; a local large VLM was not installed.
5. Bitcoin fixture: `https://bitcoin.org/bitcoin.pdf` supplies a public text-rich regression document whose transaction and timestamp diagrams exercise the selective candidate route.

The PaddleOCR-VL URL tested during validation returned 404; therefore no unsupported installation command or benchmark claim is included as an implementation requirement. The repository's existing PaddleOCR links are retained as upstream references only.

Verification evidence

Deterministic source checks completed:

```
cargo fmt --all                            PASS
cargo check                                PASS
cargo test --all-targets                   19 passed
cargo clippy --all-targets --all-features -- -D warnings  PASS
python3 -m py_compile helper               PASS
python3 git diff --check                   PASS
cargo build --release                      PASS
```

Runtime contract test used a local fake OpenAI-compatible HTTP server and a process-local test credential. It verified:

- page rendering;
- Base64 image request;
- strict response parsing;
- source SHA-256 recording;
- atomic artifact write;
- labels, relationships, and uncertainties;
- absence of the test credential from the artifact.

The Bitcoin integration test ran nine native pages with `--visual` against that local fake endpoint:

```
pages processed       9
reconstruction errors 0
visual candidates     3
visual success        3
visual artifacts      3
```

This proves the route and artifact contract. It is not evidence of production model quality because the endpoint was deliberately fake. A real cloud smoke test requires a credential supplied at execution

time; no credential was available in the current process, so no fabricated production result is reported.

Release gates

Before commit and push:

```
cargo fmt --all -- --check
cargo test --all-targets
cargo clippy --all-targets --all-features -- -D warnings
cargo build --release
git diff --check
python3 scripts/check_repo_hygiene.py
pandoc docs/refactor-plan-vlm.md -o docs/pdf2md-vlm-refactor-report.pdf --pdf-engine=xelatex -
pdfinfo docs/pdf2md-vlm-refactor-report.pdf
pdftotext -layout docs/pdf2md-vlm-refactor-report.pdf /tmp/pdf2md-vlm-report.txt
```

The repository must not commit PDFs, PNGs, Base64 payloads, caches, temporary outputs, or credentials unless a fixture is explicitly designated and sanitized. The generated report is an intentional deliverable; runtime Bitcoin artifacts stay outside the repository.

Ponytail decision

The shipped design is the smallest vertical slice that provides real value: candidate detection, temporary rendering, strict companion JSON, bounded cloud request, Markdown provenance, manifest coverage, and a verified XeLaTeX report. Deferred items are custom cropping, Mermaid, local VLM installation, provider abstraction, queueing, and a visual-specific cache layer. Add them only when measurements show the current route is insufficient.